

17-214/514: Agentic Software Development

Design Patterns and Tradeoffs

Hammad Ahmad and Claire Le Goues • Wednesday, September 30, 2026 • **Lecture**

Administrivia

- A3 checkpoint due tonight.
- Some A2 submissions are missing their transcripts. Please double-check to make sure you included all of your transcripts for A2.

What did we talk about last time?

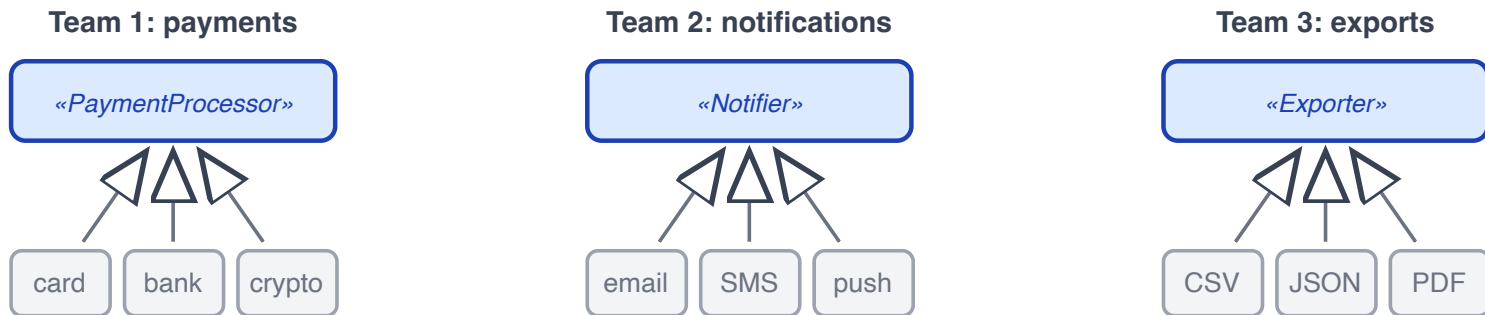
- A refactoring changes structure and keeps the observable behavior. If behavior changed, it was not a refactoring.
- Characterization tests pin what the code does today, oddities included, before we touch it.
- Direct an agent's refactor by naming it, bounding the scope, and saying what must not change.
- Refactor or regenerate is a judgment call. Trace what the change touches, then decide (coverage, age, spec quality, reach).
- Today: design patterns, and when to use one.

Learning goals for today

- Explain what design patterns are and why they exist
- Explain the Open/Closed principle and how patterns apply it
- Name the quality attributes a design decision affects, and argue a tradeoff in those terms
- Evaluate what a pattern gains and what it costs, and recognize pattern misuse
- Critique a design using pattern vocabulary, and use a pattern name as a checkable instruction to an agent

Three codebases with the same shape

- A payment router picks a processor at runtime (card, bank transfer, crypto).
- A notifier picks a channel (email, SMS, push).
- An exporter picks a format (CSV, JSON, PDF).
- Each one has variants behind a common interface, chosen at runtime.



Coincidence?

Why patterns exist

Where patterns come from

- Some problems keep coming back.
 - "The behavior varies at runtime."
 - "Something has to react to this change without being coupled to it."
 - "Someone has to decide which one gets created."
- Good solutions to them kept converging on the same shapes.
- Eventually the shapes were written down as design patterns.

What is a pattern?

PART 1

A name

a word or two we can use in a design conversation.

PART 2

A problem

that keeps recurring across projects or contexts, and is hard because several needs compete.

PART 3

A solution

a structure that solves it (roles, responsibilities, relationships).

PART 4

Consequences

what you gain and lose.

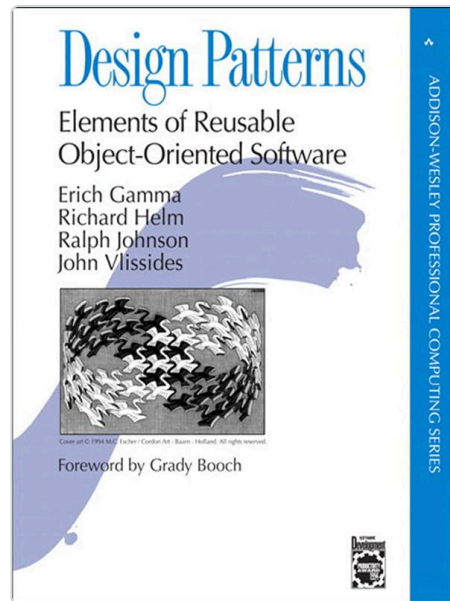
Notably, code is not on the list. A pattern is a shape to instantiate, plus a judgment about when.

What makes a problem hard?

- A design usually has to meet several needs at once.
- Take the notifier from before. It has three:
 - New channels keep being added.
 - The code that sends a message should not change each time.
 - We still want to see which channel a message used.
- They pull against each other. A pattern settles them in a given way, and its consequences say what it gave up.

The catalog

- The 1994 book **Design Patterns** (Gamma, Helm, Johnson, and Vlissides, the "Gang of Four") named 23 of them.
- Some aged well (Strategy, Observer, and Factory Method, among others). Some aged less well (Singleton, for example).
- The book's lasting contribution is arguably a shared vocabulary for design structures.



en.wikipedia.org/wiki/Design_Patterns

A working set of patterns



refactoring.guru/design-patterns/strategy

Strategy

The problem: New pricing rules keep coming, and `BookingService` should not change.

```
interface PricingPolicy {  
    int price(Booking b);  
}  
  
class FlatRate implements PricingPolicy {  
    public int price(Booking b) { return 10; }  
}  
  
class PeakPricing implements PricingPolicy {  
    public int price(Booking b) { return isPeak(b.slot()) ? 25 : 10; }  
}
```

`BookingService` holds a `PricingPolicy` and never asks which one. A new pricing rule is a new class, and the service does not change.

How does the strategy get in there?

```
BookingService service = new BookingService(new PeakPricing());
```

- Constructor injection. Whoever builds the service picks the policy, and the service never constructs its own.
- If `BookingService` calls `new PeakPricing()` inside, a test cannot easily swap it or fake it, and the pattern buys little.
- A test can hand the service a fixed-price fake the same way. That is dependency injection.



Two ways to vary a step

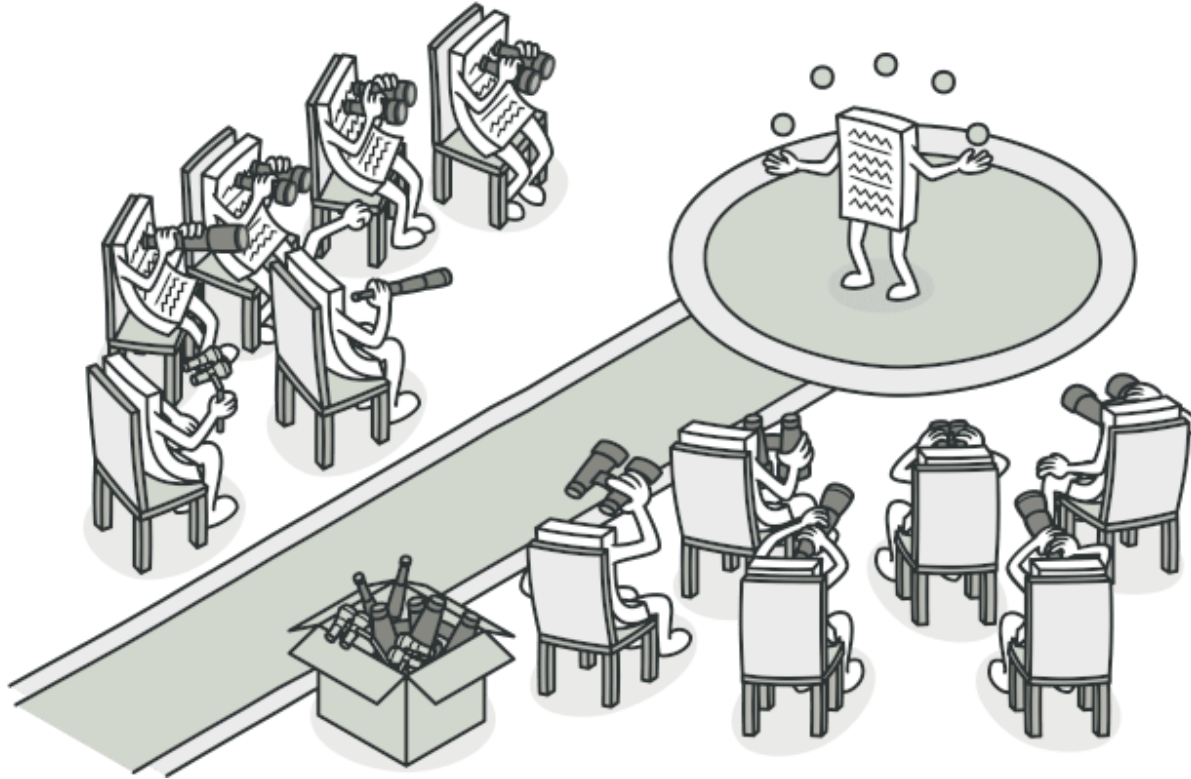
Inheritance

- A subclass **is a** kind of its parent and gets all of its methods.
- It overrides the ones it wants different.
- Coupled to the parent's insides. A change up there can reach every subclass.

Composition

- An object **has a** helper as a field and calls it.
- It exposes only what it chooses to.
- The helper can be swapped, faked, or passed in from outside.

Both rely on **polymorphism**. The call goes to whichever object is there, and the caller does not check which kind.

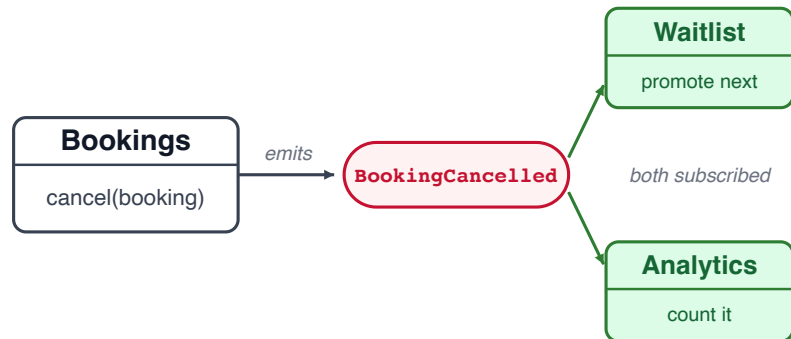


refactoring.guru/design-patterns/observer

Observer

A cancellation frees a slot. The waitlist would like to know. So would analytics, and the email digest.

- One option is that `cancel()` calls all three, and gains a call for each new reaction.
- The other is that bookings emit `BookingCancelled` and interested parties subscribe.



Bookings doesn't know who's listening.

Life without Observer

The problem: Reactions keep being added, and `cancel()` should not have to know about each one.

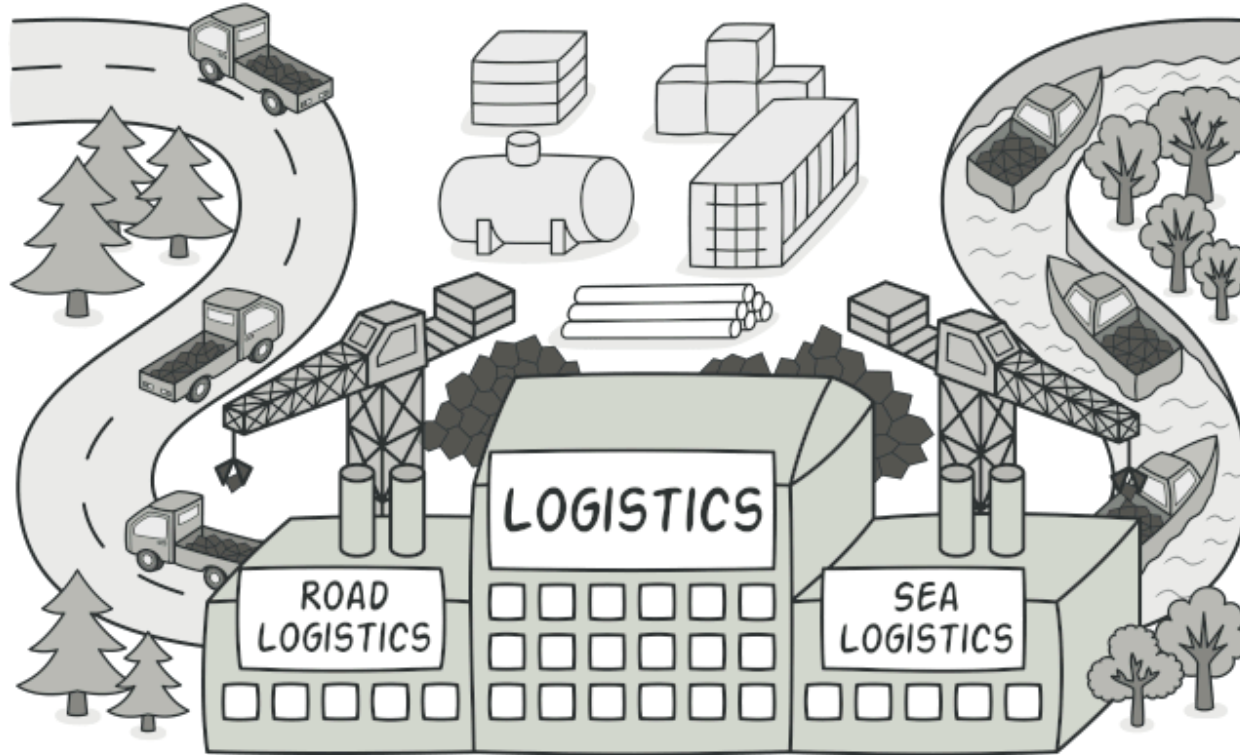
```
void cancel(Booking booking) {  
    store.remove(booking);  
    waitlist.promoteFor(booking.slot());           // knows about waitlists  
    analytics.record("cancellation", booking);     // and analytics  
    digest.enqueue(booking.user(), cancelledCopy(booking)); // and email copy  
}
```

- `cancel()` gains a call for each new reaction, and the domain now knows about email copy.
- Every new reaction means editing this working, tested method.

Wiring Observer

```
// when a BookingCancelled event happens, call this function
bookings.on(BookingCancelled.class, e -> waitlist.promoteFor(e.slot()));
bookings.on(BookingCancelled.class, e -> analytics.record("cancellation", e.booking()));
```

- Adding a subscriber does not touch `cancel()`.
- But reading `cancel()` no longer tells us what happens after a cancellation. We have to find the subscribers.
 - The order they run in is usually not written down either.
- We will revisit `BookingCancelled` later, when the system runs on several machines and this event becomes a "message".



refactoring.guru/design-patterns/factory-method

Factory

- Which notifier does this user get? Email, SMS, or a digest, depending on preferences and quiet hours.
- **The problem:** Many callers need a notifier, and the rule for choosing one should live in one place.
 - If every caller decides, the rule is duplicated, and the copies may disagree eventually.
- A factory is the one place that makes the decision. Callers ask for "a notifier for this user" and don't see the choices.

A factory for notifiers

```
function notifierFor(user: User): Notifier {  
  if (user.quietHours.activeNow()) return new DigestNotifier();  
  return user.prefersSms  
    ? new SmsNotifier(twilio)  
    : new MailgunNotifier(mailgun);  
}
```

- The quiet-hours rule lives in one place. Callers cannot forget it, because callers don't see it.
- Adding push notifications is one new branch here (and a new notifier class), instead of an edit in every caller.
- But the factory now depends on every notifier class.
 - The call site no longer shows which notifier a user gets. We have to look in the factory.

The working set

When you hear...	Consider
"the algorithm varies by X"	Strategy
"when this happens, others need to react, loosely"	Observer
"who decides which one gets created?"	Factory

Hearing the phrase tells you which pattern to consider. Whether to apply it is a design decision.

More patterns we'll see later

Pattern	What it is for	Read more
Adapter	Wraps something with the wrong interface so that it fits the one our code expects.	refactoring.guru/design-patterns/adapter
Template Method	A base class fixes the steps of a procedure, and subclasses fill some of them in.	refactoring.guru/design-patterns/template-method
Facade	One simple entry point in front of several subsystems.	refactoring.guru/design-patterns/facade
Decorator	Wraps an object to add behavior, and keeps the same interface.	refactoring.guru/design-patterns/decorator

The principle underneath

SOLID: The Open/Closed principle

Open/Closed principle (OCP): Software should be **open for extension** and **closed for modification**.

- Add new behavior by adding new code rather than by editing working code.
- Editing working code can re-open verified behavior.
- Extending through a designed extension point mostly risks the new part.

Many patterns are extension points

- A new pricing rule is a new `PricingPolicy` class, and `BookingService` is untouched (Strategy).
- A new reaction to a cancellation is a new subscriber, and the publisher is untouched (Observer).
- A new notification channel is one new branch in the factory, and callers are untouched (Factory).
- Strategy and Observer are extension points, places where new behavior can be added without editing what is there. The Factory still takes an edit in one place.

Open along which axis?

- No design is open to every extension. Each extension point usually serves one axis of change.
 - `PricingPolicy` takes new pricing rules, but does nothing for booking rules.
- So applying OCP means guessing which change is coming, and the principle does not make that guess for us.
- "Make everything extensible" is speculative generality.
- Also: "Closed" is a goal rather than a guarantee. Some changes will still edit the core, which is what the suite and the pins are for.

Extension points are also injection points

- The place a new strategy plugs in is the place a fake plugs in.
 - Think of it as controllability through structure instead of through test setup.
- Generally, a pattern you cannot test through deserves a second look.

Every pattern is a tradeoff

Indirection is not free, BTW

- A pattern usually adds indirection. The behavior you are looking for may no longer be where you are looking.
- "Where is the price computed?" is now a two-hop question through an interface.
- A pattern usually makes the code harder to follow now and easier to change later, which is worth it only if the change comes.



programmerhumor.io

Quality attributes

- Recall the "-ilities" from Week 3.
 - **Modifiability.** How much has to change for the next likely change?
 - **Testability.** Can a test reach and control the part that matters?
 - **Understandability.** How much has to be read to follow one request?
 - **Performance.** How much work, memory, or waiting per operation?
 - **Reliability.** What happens when a part fails?

Attributes pull against each other

A design that gains...	often pays in...
modifiability (an extension point)	understandability (one more hop)
performance (a cache, a denormalized copy)	modifiability and reliability (two things to keep in sync)
reliability (retries, fallbacks)	understandability and performance
testability (an injection point)	understandability (the wiring moves up a level), a price worth paying generally

No design wins on every attribute. Arguing a tradeoff means saying which attribute we gave up.

The same rule with and without Strategy

An if

```
int price(Booking b) {  
    return isPeak(b.slot()) ? 25 : 10;  
}
```

Understandability: one glance.

Modifiability: a third rule edits this method.

Testability: through `price()`.

Strategy

```
interface PricingPolicy { ... }  
class FlatRate implements ... { ... }  
class PeakPricing implements ... { ... }  
// plus the wiring
```

Understandability: two hops.

Modifiability: a third rule is a new class.

Testability: a fake policy.

Which would you pick with two rules?

With five that change on their own schedules?

Should I use a pattern here?

It depends.

On what?

- Is the pattern's problem present today, or coming up soon?
 - For Strategy, how many variants exist today, and do they change independently?
- If not, can the pattern wait?
 - Usually yes, when the code is yours and its behavior is pinned. Adding the pattern later is a refactor.
- When can it not wait?
 - When someone who cannot edit your code would have to change with it (a published API, a plugin, stored data).
- Until then, the simpler design is usually enough.

Misuse and soup

Pattern misuse

- Pattern **misuse** is a good structure applied where its problem is absent. For example, a Strategy with one strategy, or a Factory for a class with one constructor call.
- An **anti-pattern** is a common design that looks reasonable and causes trouble (the God class, for example).
- Simple enough critique of pattern misuse: "not here".

Pattern soup

```
class NotifierFactory {  
    createNotifierStrategy(kind: string): NotifierStrategy {  
        const events = new NotifierEvents();  
        events.subscribe(new EmailSentObserver());  
        return new EmailNotifierStrategy(new EmailClient(), events);  
    }  
}
```

Three patterns to send one email. Each layer is a place for a bug to hide and a reader to get lost. Does every layer solve a problem we currently have?

Pattern soup in the wild

Class `AbstractSingletonProxyFactoryBean`

```
java.lang.ObjectⒺ  
  org.springframework.aop.framework.ProxyConfig  
    org.springframework.aop.framework.AbstractSingletonProxyFactoryBean
```

All Implemented Interfaces:

```
SerializableⒺ, Aware, BeanClassLoaderAware, FactoryBean<ObjectⒺ>, InitializingBean
```

Direct Known Subclasses:

```
CacheProxyFactoryBean, TransactionProxyFactoryBean
```

```
public abstract class AbstractSingletonProxyFactoryBean  
  extends ProxyConfig  
  implements FactoryBean<ObjectⒺ>, BeanClassLoaderAware, InitializingBean
```

Convenient superclass for `FactoryBean` types that produce singleton-scoped proxy objects.

Manages pre- and post-interceptors (references, rather than interceptor names, as in `ProxyFactoryBean`) and provides consistent interface management.

docs.spring.io, spring-framework javadoc, AbstractSingletonProxyFactoryBean

A real class in a real framework. *Some* of its layers solve problems a framework has. A name like it in your own code is worth a second look.

Unwinding the soup

- What did the requirements ask for? Email today. Maybe SMS someday.
- A sufficient design is `sendEmail(to, msg)`, plus at most one interface at the vendor boundary.
 - That interface does solve a real problem (tests can use a fake in its place).
- The rest can go.

Patterns in the wild

- A comparator you pass to a sort is a **Strategy**.
- An `addEventListener` callback is an **Observer**, and so is a React subscription.
- A database driver is usually handed out by a **Factory**.
- You have probably been using patterns for a while.

Patterns and agents

A pattern name as an instruction

- "Extract a PricingPolicy Strategy from BookingService and keep the flat and peak variants" is one sentence.
- It names the roles, the structure, and where the extension point goes.
- It is also checkable. You can verify that the diff matches the named shape, which is harder with a vague prompt.

Agents reach for patterns too

- Ask for a booking feature and you might receive a `BookingStrategyFactoryProvider`.
- This is speculative over-abstraction.
 - This often happens for the usual reasons (an underspecified request, and structure that costs the agent nothing to generate, etc.).
- How often it happens varies by model and release. As of right now, it is common enough to keep an eye out for.
- The question you should ask is "which problem does this pattern solve here?"

Critique with pattern vocabulary

- Smells are vocabulary for what might be wrong.
- Patterns are vocabulary for what should be there, and for what is there without a reason.
- "The notifier should be a Strategy, with three variants changing independently" is a critique someone can argue with (unlike "this feels over-engineered").
- There is no correct design, but there are many incorrect ones.
 - Pattern names let you say which kind you are looking at.

Which of these patterns are justified?

An agent's plan for booking notifications, where email is the only channel today:

1. `Bookings` emits `BookingCancelled` and a `NotificationService` subscribes.
(Observer)
2. A `NotifierFactory` returns the right `Notifier` for a user. (Factory)
3. A `MessageFormatter` interface with one implementation, `PlainTextFormatter`.
(Strategy)



Canvas → Quizzes →
Lecture 10 In-Class Quiz

For each, is there a problem it solves today? Would you unwind it, or keep before its problem arrives?

SMS ships next month. Do your answers change?

Patterns get bigger

- Today's patterns arrange classes. The same idea exists at system scale (layered architectures, event buses, pipelines), one level up.
- We will revisit patterns for architecture in a few weeks.
- The same rules/reasoning apply there.

The A3 checkpoint is tonight

- `AUDIT.md` at the root of your repository by 11:59 pm.
 - What each new feature affects, two smells, what makes version 1 easy or hard to keep, and evolve or regenerate.
 - Submit the commit link on Canvas.
- The audit is design critique of your own system. Name files and line ranges, and give a reason that is true of your system rather than of systems in general.
- Commit to a road tonight.
 - The addendum later at the assignment deadline should talk about what the audit got right and wrong.

Summary

- A pattern has a name, a problem, a solution, and consequences. It is a shape to instantiate, and a judgment about when.
- The working set is Strategy, Observer, and Factory. You have probably used them already.
- The **Open/Closed** principle says to extend through designed extension points instead of editing working code. Many patterns are extension points, and an extension point is usually an injection point too.
- A pattern usually costs some indirection. Argue the tradeoff in attribute terms, and let the problem justify the pattern. Two stable variants usually deserve an `if`.
- A pattern name is a checkable instruction to an agent. When an agent adds a pattern, check that it solves a problem you have.

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6 A3 chpkt	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
---------------------	-----------	---------------------	-----------	--------------------------	-----------------------	---------------------	-----------	---------------------	------------------	------------	----------------------	------------	----------------------

- **Friday, Lab 6:** An API contract under pressure. One safe change, one breaking change, and a consumer you do not control.
- **Monday:** APIs, libraries, and frameworks. Contracts for strangers, human and agent, and more on SOLID.
- **Next Wednesday:** A3 due, 11:59 pm.